Introduction to Pandas

Agenda

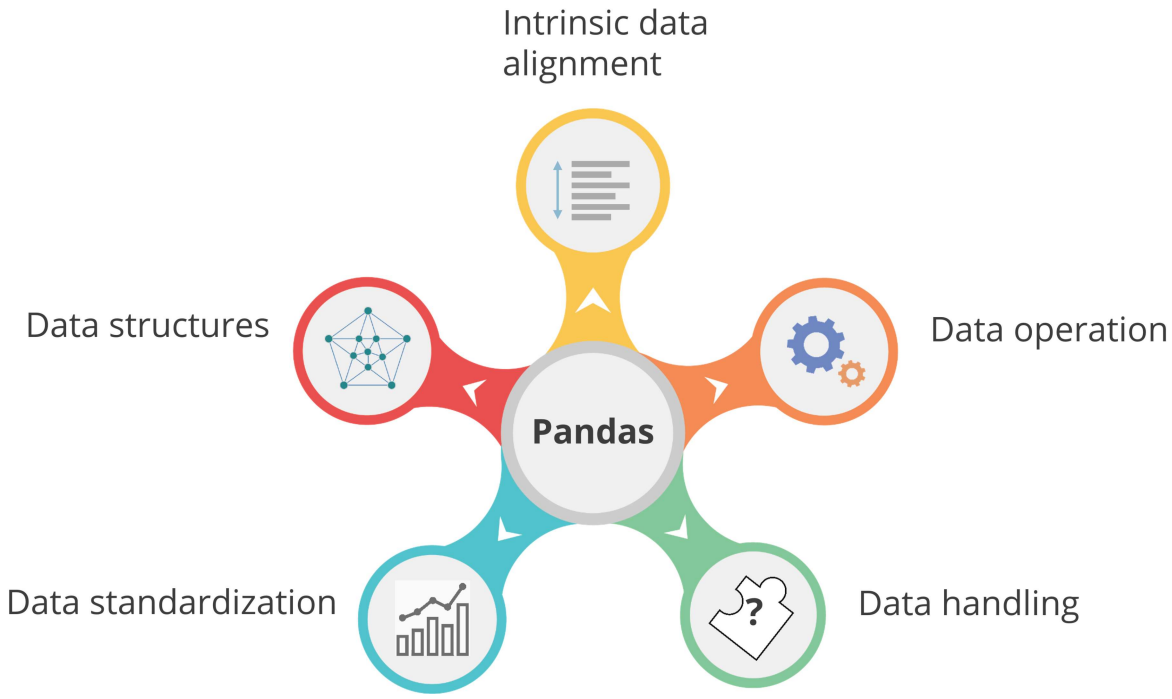
- Fundamentals of Pandas
  - Purpose of Pandas
  - Features of Pandas
- Data Structures
- Introduction to Series
  - Creating and Accessing Pandas Series Using Different Methods
  - Basic Information in Pandas Series
  - Operations and Transformations in Pandas Series
  - Querying a Series

1. Fundamentals of Pandas

Pandas is an open-source library built on top of NumPy and is used for data manipulation.

- It introduces data structures like DataFrame and Series that make working with structured data more efficient.

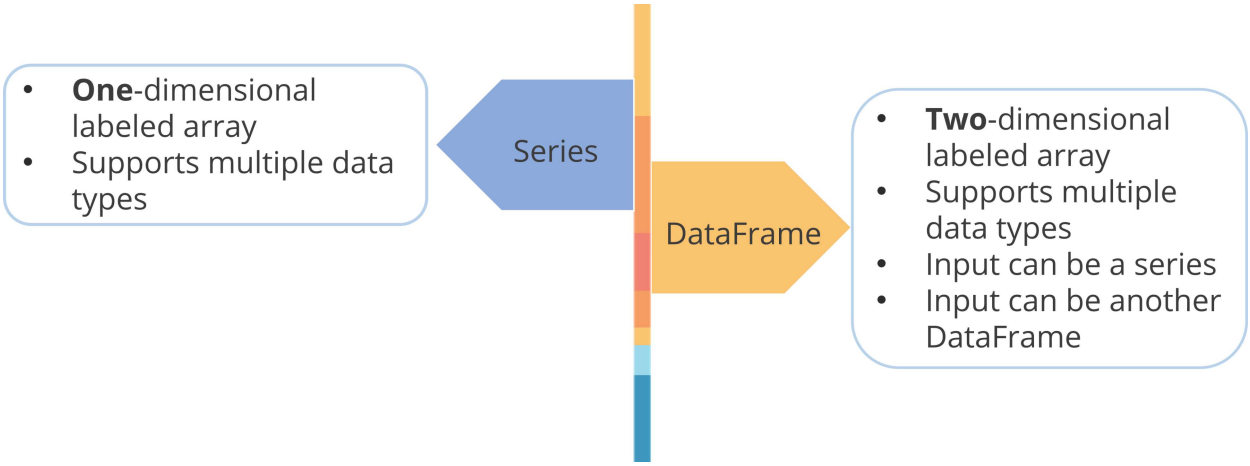
1.1 Purpose of Pandas



1.2 Features of Pandas

2. Data Structures

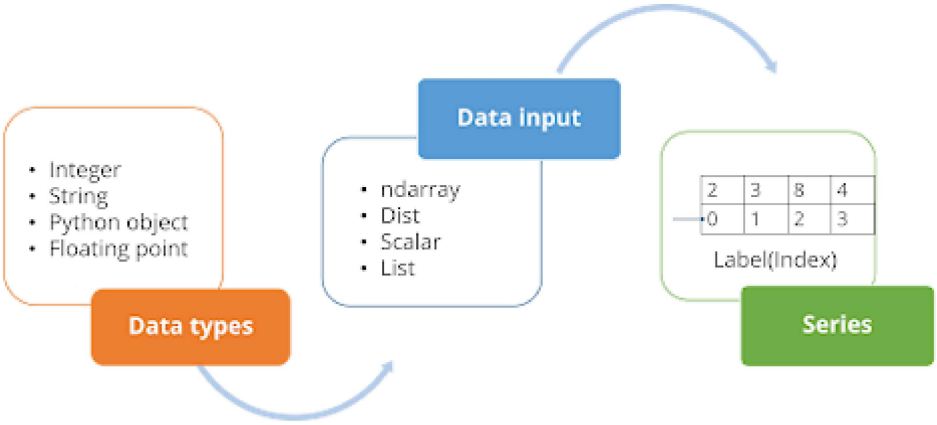
The two main libraries of Pandas data structure are:



3. Introduction to Series

A Series is a one-dimensional array-like object containing data and labels or index.

It can be created with different data inputs:



3.1 Creating and Accessing Pandas Series Using Different Methods:

```
import pandas as pd

# Creating a Pandas Series from a list
data = [1, 2, 3, 4, 5]
series = pd.Series(data)

# Creating a Pandas Series with a specified index
index = ['a', 'b', 'c', 'd', 'e']
series_with_index = pd.Series(data, index=index)

# Creating a Pandas Series from a dictionary
data_dict = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

```
series_from_dict = pd.Series(data_dict)

# Accessing data in a Series
print(series[2])  # Accessing element at index 2
print(series_with_index['b'])  # Accessing element with index 'b'
```

```
3
2
```

3.2 Basic Information in Pandas Series

These functions collectively help analysts summarize and understand the characteristics of the data, facilitating effective data exploration and analysis.

```
# Return the first n rows
first_n_rows = series.head(3)

# Return the last n rows
last_n_rows = series.tail(3)

# Return dimensions (Rows, columns)
dimensions = series.shape

# Generate descriptive statistics
stats = series.describe()

# Return unique values
unique_values = series.unique()

# Return the number of unique values
num_unique_values = series.nunique()
```

3.3 Operations and Transformations in Pandas Series

Operations and transformations in Pandas Series are crucial for modifying, enhancing, and cleaning data effectively.

They provide flexibility to adapt data to specific analyses or visualizations, preparing it for meaningful insights and ensuring data quality.

```
# Element-wise addition
result_series = series + series_with_index

# Apply a function to each element
squared_series = series.apply(lambda x: x**2)

# Map values using a dictionary
mapped_series = series.map({1: 'one', 2: 'two', 3: 'three'})
```

```
# Sort the Series by values
sorted_series = series.sort_values()

# Check for missing values
missing_values = series.isnull()

# Fill missing values with a specified value
filled_series = series.fillna(0)
```

3.4 Querying a Series

Selecting and filtering data based on specific conditions is an essential aspect of querying a Pandas Series.

The following examples illustrate common querying operations that can be applied to a Pandas Series:

```
import pandas as pd

# Create a Pandas Series
data = {'a': 10, 'b': 20, 'c': 30, 'd': 40, 'e': 50}
series = pd.Series(data)

# Select elements greater than 30
selected_greater_than_30 = series[series > 30]

# Select elements equal to 20
selected_equal_to_20 = series[series == 20]

# Select elements not equal to 40
selected_not_equal_to_40 = series[series != 40]

# Select elements based on multiple conditions
selected_multiple_conditions = series[(series > 20) & (series < 50)]

# Select elements based on a list of values
selected_by_list = series[series.isin([20, 40, 60])]

# Select elements using string methods (if applicable)
string_series = pd.Series(['apple', 'banana', 'cherry', 'date', 'elderberry'
selected_by_string_method = string_series[string_series.str.startswith('b')]

# Query based on index labels
selected_by_index_labels = series.loc[['a', 'c', 'e']]

# Query based on numeric position
selected_by_numeric_position = series.iloc[1:4]

# Display the results
print("Original Series:")
print(series)
print("\nSelected greater than 30:")
```

```
print(selected_greater_than_30)
print("\nSelected equal To 20:")
print(selected_equal_to_20)
print("\nSelected not equal to 40:")
print(selected_not_equal_to_40)
print("\nSelected based on multiple conditions:")
print(selected_multiple_conditions)
print("\nSelected based on list of values:")
print(selected_by_list)
print("\nSelected based on string method (startswith):")
print(selected_by_string_method)
print("\nSelected based on index labels:")
print(selected_by_index_labels)
print("\nSelected based on numeric position:")
print(selected_by_numeric_position)
```

```
Original Series:
a    10
b    20
c    30
d    40
e    50
dtype: int64

Selected greater than 30:
d    40
e    50
dtype: int64

Selected equal To 20:
b    20
dtype: int64

Selected not equal to 40:
a    10
b    20
c    30
e    50
dtype: int64

Selected based on multiple conditions:
c    30
d    40
dtype: int64

Selected based on list of values:
b    20
d    40
dtype: int64

Selected based on string method (startswith):
```

```
1    banana
dtype: object

Selected based on index labels:
a    10
c    30
e    50
dtype: int64

Selected based on numeric position:
b    20
c    30
d    40
```

Assisted Practice

Problem Statement:

Use Pandas Series to analyze sales data for a retail store over a week and draw insights from the data.

Dataset:

Sample sales data

sales_data = [120, 150, 130, 170, 160, 180, 140]

days_of_week = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

Steps to Perform

1. Create a Pandas Series for sales data

- Use a list of daily sales figures to create a Pandas Series
- Assign days of the week as the index

2. Access and manipulate sales data

- Access sales data for specific days using index labels
- Calculate total sales for the week
- Identify the day with the highest and lowest sales

3. Basic analysis of sales data

- Calculate the average sales for the week

- Determine the days with sales figures significantly different from the average